

Latency Optimization Trajectory — VisionPsy-Nano-460M On-Device

Status: **experimental, partially validated, NOT fully shipped in the app**. Last updated: 2026-09-18

This document exists for reproducibility and future reference. It separates what has been **measured**, what has been **fixed in source but not committed**, and what has been **proposed but not built into the actual app** — those are three different states and this doc keeps them distinct on purpose.

1. TL;DR

Stage	Latency	What it reflects
0. Original, real app	~99-152s per extraction (avg ~152s over 20 logged runs, range 82-233s)	Full pipeline via <code>llama-mtmd-cli</code> , one-shot OS process per run, full-size document image. This is what the shipped app does today.
1. Root cause found (not yet fixed in app)	~90-102s of the above is process spawn + full model reload from flash, on every single run	Diagnosed in <code>latency_autopsy.pdf</code> . Fix proposed ("server mode" / keep model resident) but never implemented in <code>MainActivity.kt</code> .
2. Warm-server benchmark (test harness only)	55-76s per field (box case), 56-61s (crop case)	Built for controlled box-vs-crop accuracy testing. Uses <code>llama-server</code> (persistent process) instead of <code>llama-mtmd-cli</code> — this eliminates the reload cost from stage 0/1, but only inside the test harness , not the app.
3. Tile/upscale root cause found + fixed	14-16s per field (crop case, warm server)	This session's work. Root cause: vision preprocessing force-upscases every image (including tiny crops) to a fixed 2048px canvas before tiling. Fixed via <code>MTMD_MAX_LONGEST_EDGE=768</code> env var. Validated on 4 crops, Nano only, zero accuracy regression.

The gap that remains: nothing from stage 2 or 3 has been wired into the actual Android app. The app still runs stage 0's behavior today. Realizing "14-16s in the user's hand" requires *also* shipping the stage-1 server-mode rewrite — that part is still proposed, not built.

2. Full timeline, with evidence

Stage 0 — Baseline, real app behavior

Source: `run_log.txt` (this repo), 20 runs, PAN + Aadhaar, full document image, `llama-mtmd-cli`, CPU only, 4 threads.

n=20 min=82.4s max=232.9s avg=152.4s

Cross-checked against [latency_autopsy.pdf](#)'s own tighter isolation test on the same phone (device [RZCXA1SXH5V](#), Exynos 1380, Mali-G68, 4 threads, `--temp 0`): a single PAN extraction measured **99-111s**, broken down as:

- **~102s** — process spawn + model load + vision-encode + prefill (measured via a `-n 1` isolation run: ask for effectively zero output tokens, see what's left)
- **~12s** — an *avoidable* warm-up forward pass llama.cpp runs by default on every load (`common_init_from_params`, disableable via `--no-warmup`, byte-identical output confirmed via `--temp 0` A/B test)
- **~9s** — actual token generation

That 102s bucket was **not** further separated into "model load" vs. "vision-encode+prefill" by that report — it explicitly flags this as unmeasured (`vision-encoder token budget: UNCONFIRMED`). That ambiguity is exactly what Stage 2/3 below ends up resolving empirically.

Stage 1 — Proposed fix, not shipped

[latency_autopsy.pdf](#), "Status: fixed, proposed, unexamined" section, explicitly lists:

Keep the model resident (server mode) — *PROPOSED, UNTESTED, large change, needs a feasibility check first*. Requires confirming the patched build's server binary carries the same custom SigLIP2 mmproj support as `llama-mtmd-cli` before any of this is built — a rewrite of the inference layer, process lifecycle, and Stop/cancel handling, not a flag. Potential: 111s → ~9-20s per repeat run.

This was never implemented. `MainActivity.kt`'s `runOnePass()` / `runExtraction()` still spawn a fresh `llama-mtmd-cli` process per extraction as of this writing.

Stage 2 — Warm-server test harness (this and prior sessions)

For controlled box-vs-crop accuracy benchmarking ([aadhaar_bench_results/](#), [pan_bench_results/](#)), a `llama-server` instance was started once on-device and kept warm across all test cases, avoiding per-request reload — **inside the test harness only**. This incidentally answers Stage 1's open feasibility question ("does the server binary support our custom SigLIP2 mmproj the same way the CLI does?") — **yes, it does**, since the entire tile-reduction experiment ran against it successfully.

Pre-tile-fix, warm-server latency (from [aadhaar_bench_results/*.json](#) and [pan_bench_results/*.json](#), `latency_s` field, Nano):

Dataset	Box avg	Crop avg
Aadhaar	70.0s (n=12)	60.7s (n=12)
PAN	75.7s (n=8)	60.1s (n=8)

Flash, same conditions:

Dataset	Box avg	Crop avg
Aadhaar	61.4s (n=12)	56.5s (n=12)
PAN	74.3s (n=8)	58.2s (n=8)

This confirmed reload elimination alone was not enough — crop requests, despite sending a much smaller image, were *not* proportionally faster than box requests. That mismatch is what triggered the Stage 3 investigation.

Stage 3 — Tile/upscale root cause, found and fixed (this session)

Root cause. `mtmd_image_preprocessor_idefics3::preprocess()` in `tools/mtmd/mtmd-image.cpp` resizes every input image so its longest edge hits `hparams.image_longest_edge` — read from the model's own GGUF metadata (`KEY_PREPROC_IMAGE_SIZE`), which is **2048px**, hard-coded per model, applied unconditionally — *before* the image is sliced into tiles. A tiny crop (e.g. ~250×80px) gets force-upscaled to a 2048px canvas first, then tiled as if it were a full-resolution image. This is why crops didn't scale down: the encoder never saw the crop's real, small size.

Confirmed via source trace:

- `clip.cpp`, `PROJECTOR_TYPE_CUSTOM` case (~line 1339): sets `hparams.n_merge` and `hparams.image_longest_edge`, does **not** populate `hparams.image_res_candidates` — ruling out the LLaVA-UHD / InternVL-DHR dynamic-resolution code paths as relevant (those are for other projector types, confirmed by reading `set_llava_uhd_res_candidates()` at ~line 3059 and `set_internvl_dhr_res_candidates()` at ~line 3074 — neither is reached by this model).
- `mtmd.cpp`, `PROJECTOR_TYPE_CUSTOM` case (~line 501): confirms this model actually instantiates `mtmd_image_preprocessor_idefics3`, `slice_tmpl = MTMD_SLICE_TMPL_IDEFICS3`.
- `mtmd-image.cpp`, `mtmd_image_preprocessor_idefics3::preprocess()` (~line 1046-1118): the actual function responsible. Confirmed by reading it in full.

Dead end tested first: `MTMD_NO_UPSCALE=1`. This env var already existed in the codebase — skips the upscale entirely, uses the crop's native resolution. Tested on-device:

Crop	Before	After (<code>MTMD_NO_UPSCALE=1</code>)	Accuracy
~254×79px	52s	6.9s	Failed — "no visible text"
~292×88px	61s	1.4s	Failed — "no visible text"

Latency wins were dramatic but the model went completely blind. Root cause of *that* failure: SigLIP2's vision encoder uses fixed 16×16-pixel patches — at native crop resolution there simply aren't enough pixels per character for the patchified representation to resolve text. **Ruled out. Not usable as-is.**

The actual fix: `MTMD_MAX_LONGEST_EDGE`. Added as a new env var, additive to the existing `MTMD_NO_UPSCALE` check, letting the upscale *target* be capped at any value instead of being locked to 2048 or disabled outright:

```
// tools/mtmd/mtmd-image.cpp, inside
// mtmd_image_preprocessor_idefics3::preprocess(), immediately after the
```

```
// existing MTMD_NO_UPSCALE block:

int effective_longest_edge = static_cast<int>(hparams.image_longest_edge);
bool no_upscale = false;
if (const char * env = std::getenv("MTMD_NO_UPSCALE")) {
    no_upscale = (env[0] == '1' || env[0] == 't' || env[0] == 'T' || env[0]
== 'y' || env[0] == 'Y');
}
// Experimental: cap the upscale target below the model's default (2048)
instead of an
// all-or-nothing switch, to find a latency/legibility middle ground for
small crops.
if (const char * env = std::getenv("MTMD_MAX_LONGEST_EDGE")) {
    int custom_edge = std::atoi(env);
    if (custom_edge > 0) {
        effective_longest_edge = custom_edge;
    }
}
```

Tested at `MTMD_MAX_LONGEST_EDGE=768`: tile count for a typical crop drops to **3 tiles** (down from a larger grid at the 2048px default). This has a compounding effect — fewer tiles means less vision-transformer compute *and* fewer resulting vision tokens for the LLM to prefill, so both stages shrink together.

Validated results (Nano, warm server, 4 hand-picked crops spanning both document types):

Crop	Field	Before	After (768px)	Accuracy
Samad's PAN card	pan_number	~52s	14-16s	Correct, unchanged
Sapna's Aadhaar	address	~61s	14-16s	Correct, unchanged
Aniz's PAN card	name	~55s	14-16s	Correct, unchanged
Jasjot's Aadhaar	name	~58s	14-16s	Correct, unchanged

Net effect: **~73-77% latency reduction on an already-warm server**, zero accuracy regression on every case tried so far.

Interestingly, this lands close to the *original* `latency_autopsy.pdf`'s speculative estimate for server mode alone ("111s → ~9-20s per repeat run") — that estimate didn't anticipate the fixed-upscale tiling problem, which is likely why the actually-measured warm-server numbers in Stage 2 (55-76s) came in much higher than predicted. Fixing the tiling problem is what closes that gap.

3. Current repo state (important — check before building on this)

 **Branch mixup found while writing this doc.** The inner repo (`qvac-visionpsy-nano`, a separate git repository from the outer project repo) reflog shows:

```
4effbda HEAD@{0}: checkout: moving from experiment/reduce-vision-tiles to
main
```

```
4effbda HEAD@{1}: checkout: moving from main to experiment/reduce-vision-
tiles
4effbda HEAD@{2}: clone: from https://github.com/tether-ai-research/qvac-
visionpsy-nano.git
```

The `experiment/reduce-vision-tiles` branch **was** created as intended, but something later checked back out to `main` (same commit, no branch-side commits made). The **MTMD_MAX_LONGEST_EDGE** fix **currently sits as an uncommitted working-tree change on `main`**, not on the experiment branch:

```
$ git status --short
M llama-cpp-inference/llama.cpp-custom/tools/mtmd/mtmd-image.cpp
$ git branch --show-current
main
```

Before committing anything, switch back to the experiment branch (or recreate it from current `main` + stash the diff) so this work stays isolated from `main` as originally intended:

```
cd /home/kaarunya/Downloads/model_testing/home/aiteam/model_testing/qvac-
visionpsy-nano
git stash
git checkout experiment/reduce-vision-tiles # or: git checkout -b
experiment/reduce-vision-tiles
git stash pop
git add llama-cpp-inference/llama.cpp-custom/tools/mtmd/mtmd-image.cpp
git commit -m "mtmd: add MTMD_MAX_LONGEST_EDGE to cap vision preprocessing
upscale target"
```

4. Reproduction steps

4.1 Rebuild after a source change

`cmake` itself was found to be missing from this machine (likely cleared between sessions) — but the Makefiles CMake generated in an earlier session still exist in `build-android-server/`, so incremental rebuilds don't need `cmake` again. The NDK's own bundled `make` drives them directly:

```
cd /home/kaarunya/Downloads/model_testing/home/aiteam/model_testing/qvac-
visionpsy-nano/llama-cpp-inference/llama.cpp-custom/build-android-server
/home/kaarunya/Downloads/model_testing/home/aiteam/model_testing/toolchain/
android-ndk-r27d/prebuilt/linux-x86_64/bin/make -j4 mtmd
```

This only recompiles the changed `.cpp` file and relinks `libmtmd.so` (~15s), not a full rebuild. Confirm output size looks sane before pushing (observed: 14,388,768 bytes for the version with this fix).

4.2 Deploy to device

```
adb push build-android-server/tools/mtmd/libmtmd.so  
/data/local/tmp/server_test/
```

4.3 Run with the fix

Kill any running server, then relaunch with the env var set, using whatever the existing server invocation already is (model paths, port, thread flags unchanged) — just prefix it:

```
adb shell "cd /data/local/tmp/server_test && MTMD_MAX_LONGEST_EDGE=768  
./llama-server <existing flags>"  
adb forward tcp:<port> tcp:<port>
```

Omit the env var entirely to fall back to stock 2048px behavior. Set `MTMD_NO_UPSCALE=1` instead to reproduce the confirmed-broken no-upscale dead end (do not ship this — kept here only so it isn't accidentally re-tried as if untested).

4.4 Score a test case

Use the existing terminal reporting tool against any results JSON in `aadhaar_bench_results/` or `pan_bench_results/`:

```
python3 terminal_report.py Nano:aadhaar_bench_results/nano_results.json  
Flash:aadhaar_bench_results/flash_results.json
```

5. What is NOT yet validated

- **Only 4 crops tested** with the 768px fix, **Nano only**. Not yet run against the full 24-image Aadhaar set or 16-image PAN set, and not yet tested on Flash at all.
- **No accuracy-degradation threshold found below 768px**. Only two data points exist: complete failure at no-upscale (~0px effective), full success at 768px. A requested test at ~500-640px (to find where legibility actually starts to break) was interrupted by device availability and has not run yet. Given the base tile size is 512px, values near 500-512 are the likeliest place for a real boundary to appear — worth treating any result there with extra scrutiny, not just accepting "still works."
- **Fix not committed to git** (see §3).
- **Fix not integrated into the actual app**. The Android app's inference path (`MainActivity.kt`) still uses `llama-mtmd-cli` one-shot-per-run. Neither the Stage 1 server-mode rewrite nor the Stage 3 tile cap have been wired into it. Realizing this latency improvement for an actual user requires that integration work, which has not started.
- **Cold-start / model-load time has never been isolated on its own** for the server-mode path specifically — Stage 0's ~102s bucket bundles process spawn
 - model load + vision-encode + prefill together (explicitly flagged as unseparated in [latency_autopsy.pdf](#)). We know Stage 3's 14-16s excludes reload (warm server), but we

don't have a clean "server cold boot to first request" number on this build.

- A separate Claude Code session was independently handed this same investigation in parallel ([qvac-visionpsy-nano](#) VSCode window) — its findings, if any, have not been reconciled with this document.
-

6. Source data references

- [run_log.txt](#) — Stage 0 raw latency log (20 runs, full document, real app CLI path)
- [latency_autopsy.pdf](#) — Stage 0/1 root-cause investigation and Stage-1 proposal
- [aadhaar_bench_results/{nano,flash}_results.json](#) — Stage 2 warm-server benchmark, Aadhaar
- [pan_bench_results/{nano,flash}_results.json](#) — Stage 2 warm-server benchmark, PAN
- [terminal_report.py](#) — reusable accuracy/latency reporting tool for the above
- [home/aiteam/model_testing/qvac-visionpsy-nano/llama-cpp-inference/llama.cpp-custom/tools/mtmd/mtmd-image.cpp](#) — Stage 3 fix (uncommitted, currently on [main](#) — see §3)